Assignment #4: Ping

Goal

In this assignment, you will deepen your understanding of network protocols by implementing a program in C++ that replicates the functionality of the “ping” command. You will use the ICMP protocol to implement ping, rather than TCP or UDP.

Ping is an application that tests whether a particular host is reachable across an IP network. It provides some latency information as well. Our Ping will work by sending an ICMP Echo packet to the destination and then listen for an ICMP Echo Reply (aka “Pong”).

Your program will take an IP address or domain name as an argument and send an ICMP Echo packet to the address roughly once per second. The program will receive the ICMP Echo Reply packets – or note their absence! It will then print out the results – including the payload bytes returned, the round-trip time in milliseconds, and the TTL value from the reply.

Technology

An initial project with some scaffolding will be provided for you. This project is designed for Visual Studio 2022. Your submission will be tested on a DigiPen lab-machine equivalent: Windows 10 or 11 and latest update of Visual Studio 2022.

You can download the initial project from
https://github.com/DigiPen-Network-Classes/digiping

Your program *should not use the Windows implementation of IcmpSendEcho* – you should NOT use the definitions from icmpapi.h, iphlpapi.lib, etc. Instead we’ll use RAW sockets, setting the data directly, which means that your program will require Administrator privileges to run.

Programming

Download the skeleton of the assignment from GitHub. This has some code to handle ctrl-c to exit the program, among other things.

Example Usage and Output

Note: your actual results (timing, etc) may be different due to network conditions and other factors.

```
> ping 127.0.0.1

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time=0ms ttl=128
Reply from 127.0.0.1: bytes=32 time=0ms ttl=128
<ctrl-c>
```

Another Example

```
> ping www.google.com

Pinging www.google.com with 32 bytes of data:
Reply from 142.251.215.228: bytes=32 time=16ms ttl=114
Reply from 142.251.215.228: bytes=32 time=32ms ttl=114
Reply from 142.251.215.228: bytes=32 time=15ms ttl=114
<ctrl-c>
```

And An Example of Failing to Respond:

```
> ping cs260.meancat.com

Pinging cs260.meancat.com with 32 bytes of data:
cs260.meancat.com: Request timed out.
cs260.meancat.com: Request timed out.
<ctrl-c>
```

Tasks

You will add code to initialize the WinSock library and create a RAW socket:

```
SOCKET sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
```

You'll then need to determine the address we're sending to. First check to see if the string represents an IP address, using inet_pton(). If that fails, attempt to resolve the name using getaddrinfo(). If the name cannot be resolved to an address, print a helpful error message and exit.

Note that ICMP does not use port numbers – those fields belong to the UDP and TCP headers.

Use Non-Blocking Sockets

We are not guaranteed to get a response from our ping packets – the host might be down, may not respond, or other issues. To keep the program from hanging forever waiting for a response, you will need to set the socket to be non-blocking and handle WSAEWOULDBLOCK, timeouts, etc. in an appropriate manner.

While(true)

In an infinite loop, your program will create an ICMPPacket, using the process id as the packet id and an incrementing sequence number. See the ICMP definitions (below) for more information on how to put this together.

This loop will send an ICMP packet to the destination, wait for a maximum (roughly) of 1 second for a response, then handle the response as indicated. The program should then Sleep(1000) for a second and then return to the top – sending another packet. Each ICMP packet sent should use the Process ID as the packet id and an incrementing value (1…2..3…) for the sequence number – failing to do this may result in errors and strange results!

Sending the Packet

Send the packet to the destination using sendto(). If there is an error sending (other than WSAEWOULDBLOCK, etc.) print a helpful error message and exit.

Waiting for a Response

The program should call recvfrom() and determine if a response was received. If not, it should Sleep() for a short duration (10 ms) and then try again. If the elapsed time exceeds the TIMEOUT_MS value (1000 ms), then the program should print a message that the request timed out, but should NOT terminate – instead, return to the top of the loop.

If you get a response …

The output of the response should include:

- The address 'from' the response (NOT the address you sent to!)
- The number of data bytes received in the response (NOT the number of bytes you sent!)
    - This is the payload of the ICMP packet and must not include the size of the IPv4 or ICMP headers. Just the data returned.
- The number of milliseconds between send and receive time
- The Time to Live value of the response (again, not the value you sent, the one you got back)

Because this is a RAW socket, the receive buffer will contain the entire packet - including the IPv4 and ICMP headers. You can retrieve the TTL from the IPv4 header.

You'll also need to find the length of the IPv4 header (see the diagram, below) – do not hard code this and assume all IPv4 headers will be the same length!

The number of bytes returned should be calculated by examining the total number of bytes returned, minus the IPv4 header, minus the ICMP header.

If you don't get a (successful) response …

Your program might get no response, if the destination is not online or otherwise not responding to PING packets. You can test this using the "cs260.meancat.com" server – it will not respond to ICMP pings.

Your response was "No"

Your program might get a response of a different type than ECHO_REPLY. Your program should handle the cases where the packet type received is:

- ECHO_REPLY (i.e., successful output)
- DESTINATION_UNREACHABLE
    - Print "Destination unreachable, code=" and the value of the code field returned
- TIME_EXCEEDED
    - Print "Time exceeded, code=" and the value of the code field
- (anything else)
    - "shouldn't happen", but in this case just print out the type and code received.

On Ctrl-C, Clean up and Exit

When ctrl-c is pressed, the value of g_running will be set to false, exiting the big loop. Your program will close the socket and shut down the Winsock library in the usual way.

Testing

You should get your program working vs localhost (127.0.0.1) first, then try other addresses. All configurations of your solution will be tested – including Debug and Release, and 32- and 64-bit (if supplied). These will be checked for errors, warnings, etc.

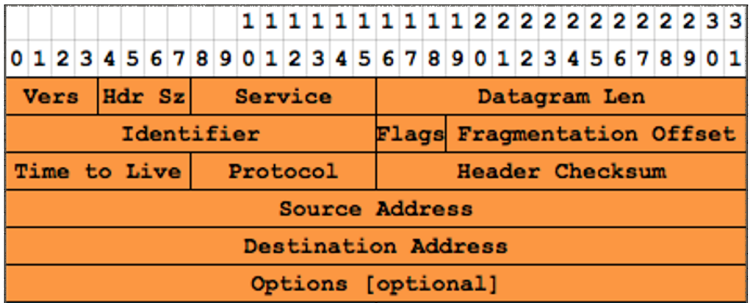
The program will be tested with both IP addresses and domain names:

- 127.0.0.1
- 1.1.1.1
- www.google.com
- cs260.meancat.com

It will also be tested vs. IP addresses that are not routable and domain names that do not exist. You must make a reasonable attempt at covering all error conditions: "ping alskdfjals" should print some reasonable error message, and so on.

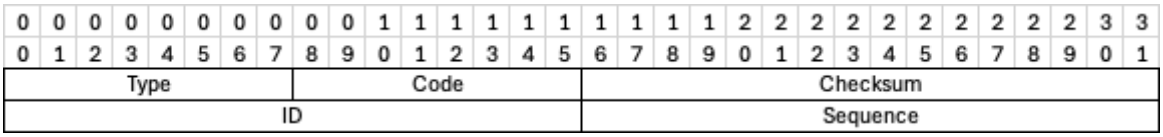
Your timing should be reasonably close to accurate, however we are not looking for or expecting better resolution than +/- several milliseconds. Using the values from GetTickCount64() should be good enough for our needs.

Internet Protocol v4 (IPv4)



Internet Control Message Protocol (ICMP)

The ICMP header starts after the IPv4 header.



Fields

- Type – the ICMP type value
- Code – a subtype to the given Type
- Checksum – Error checking data calculated from the ICMP header plus the payload, assuming a "0" for checksum. (The calculation code is provided for you.)
- ID – an identifier, should be returned in case of echo reply
- Sequence – a sequence value, should be returned in reply

Echo Request

An ICMP Echo Request sends data that the sender is expecting to see in the response ("Pong").

- The Type must be 8
- Code must be 0
- Identifier and Sequence are set by the caller. The sequence number should be incremented for each packet.
- The data received by the echo request must be entirely returned in the echo reply.

Echo Reply

- The Type must be 0
- The code must be 0
- Identifier and Sequence number should be equal to the Echo Request
- The Data should be replied entirely

Submission

Your project should be named DigiPing and basically resemble the initial program from github. You should not do anything fancy with paths or copying binaries or using cmake, etc. It should be buildable "as is" using Visual Studio 2022.

You should zip up the solution, project, and source files – but DO NOT include extra directories such as an extra top level directory, and DO NOT include build product (debug/release builds and folders).

You may include a README that contains any comments or questions – anything you think I should know about. This is optional.

Submission File Name

You must name the archive YourAlias-CS260-DigiPing.zip where "YourAlias" is the username used for Moodle.

Grading

From a base score of 100, you'll lose points for the following errors:

- -100 if your program cannot be compiled
    - Do not rely on external tools like cmake!
- -70 if your program does not send and receive ICMP packets correctly
- -30 if your program does not report destination, TTL, data size, milliseconds, etc. correctly
- -30 if your program does not correctly report non-reachable / non-pingable hosts
- -15 if your program does not report other ICMP responses correctly
- (Various) Other penalties will be assessed depending on the accuracy of the output, handling error conditions correctly, and other programming mistakes.

Code Quality

Your program should look "finished" and presentable – no large blocks of commented code; no excessive "DEBUG PRINT" output. Penalties will be assessed based on the severity of the presentation.

Your code must compile on all platforms and configurations included with the Solution, with no warnings at the default warning level of 3. Warnings will incur a 10-point penalty. Do not change the warning level for the assignment (unless otherwise specified) -- this includes Code Analysis warnings as well as build output, but not tasks or hints)

Submission Penalties

- (-10) if your submission is named incorrectly

- (-10) if your output is named incorrectly
- (-10) if your submission includes additional files